

Design Patterns for Data Ingestion

Comprehensive Guide to Ingestion Patterns with Production Examples

Table of Contents

- 1. [Introduction](#)
- 2. [Change Data Capture \(CDC\)](#)
- 3. [ETL vs ELT Patterns](#)
- 4. [Event-Driven Architecture](#)
- 5. [Micro-Batching Pattern](#)
- 6. [Stream Processing Patterns](#)
- 7. [Data Lake Ingestion Patterns](#)
- 8. [Real-Time Analytics Pipelines](#)
- 9. [Multi-Source Aggregation](#)
- 10. [Pattern Selection Guide](#)

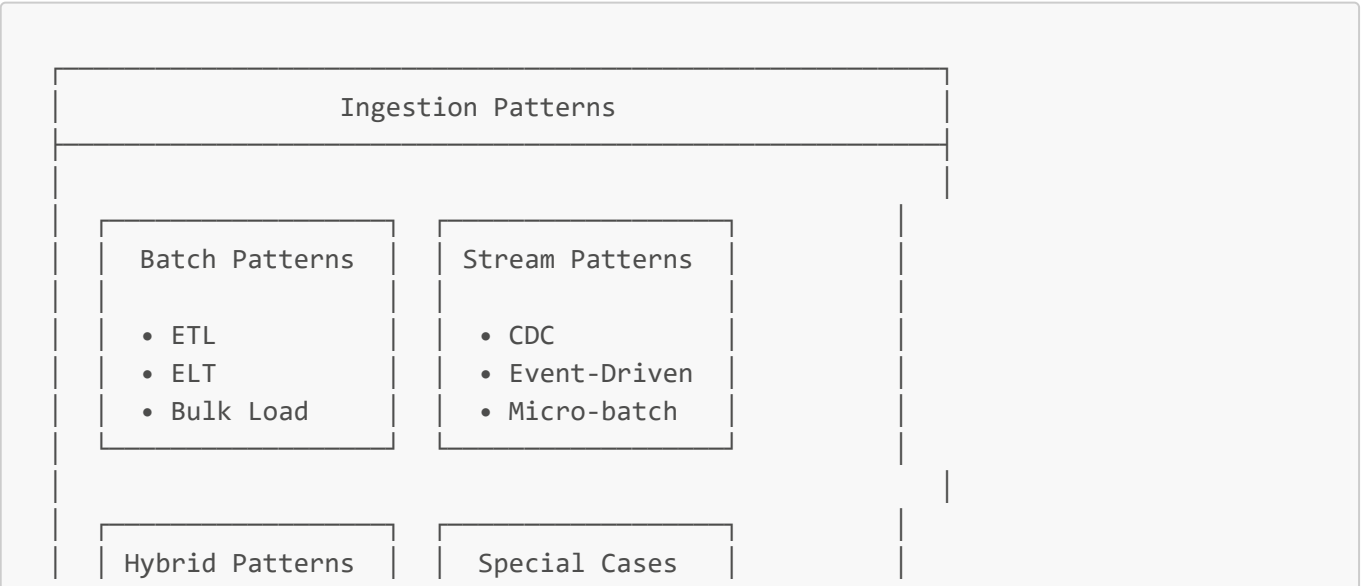
1. Introduction {#introduction}

What are Data Ingestion Patterns?

Data ingestion patterns are proven, reusable solutions to common data integration challenges. Each pattern addresses specific requirements around:

- **Latency:** How fast data needs to be available
- **Volume:** Amount of data to process
- **Variety:** Types and formats of data sources
- **Reliability:** Guarantees around data delivery
- **Cost:** Infrastructure and operational expenses

Pattern Categories



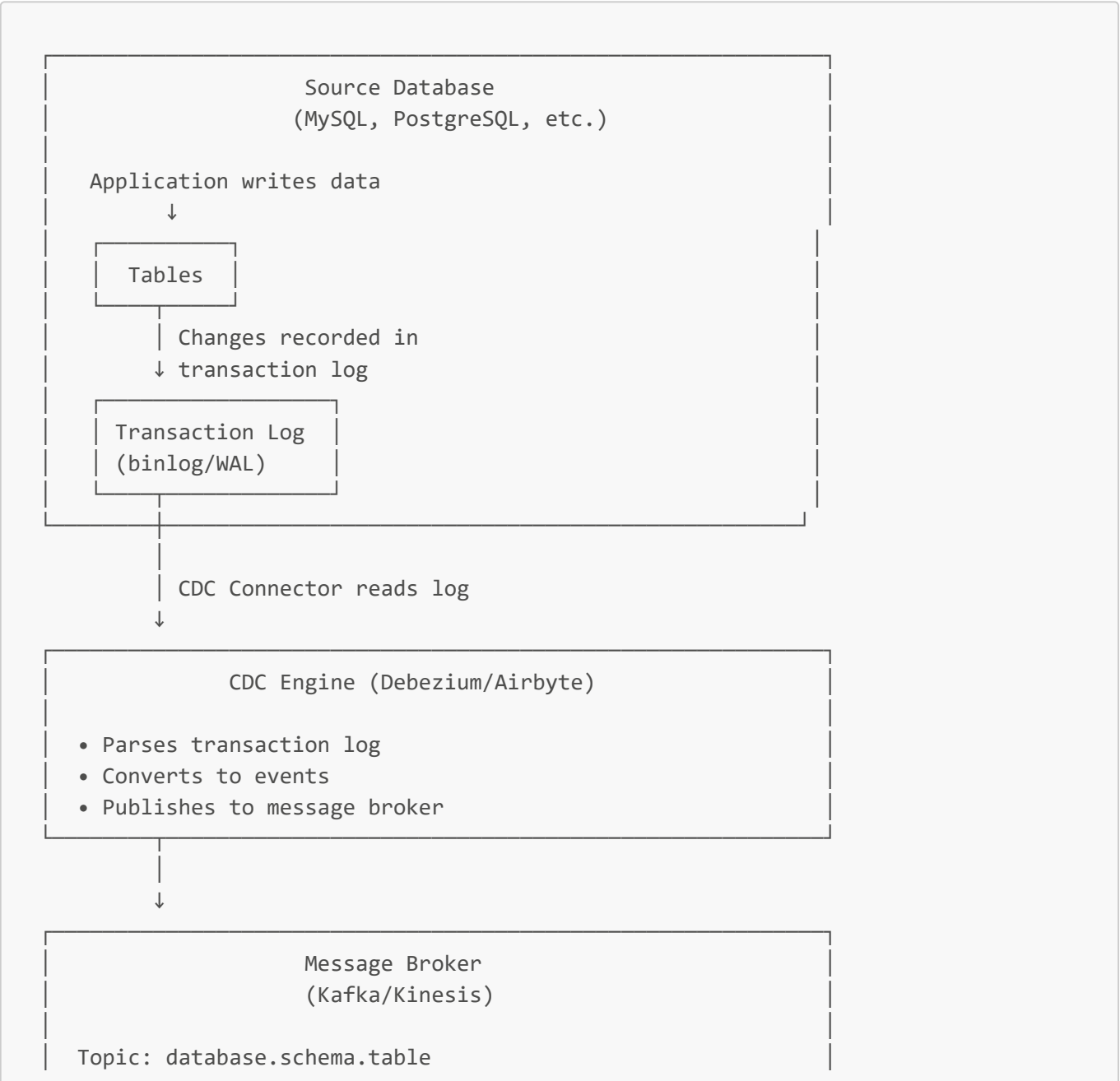
- Lambda
 - Kappa
 - Multi-layer
- Multi-source
 - API polling
 - File watching

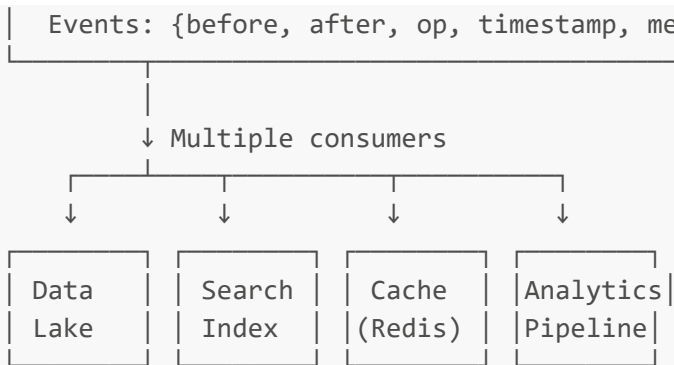
2. Change Data Capture (CDC) {#cdc-pattern}

Overview

Change Data Capture captures row-level changes (INSERT, UPDATE, DELETE) from databases in real-time by reading transaction logs, enabling event-driven architectures without impacting source database performance.

Architecture





Implementation Options

Option 1: Debezium (Most Popular)

MySQL CDC with Debezium:

```

{
  "name": "mysql-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.mysql.MySqlConnector",
    "tasks.max": "1",
    "database.hostname": "mysql.prod.company.com",
    "database.port": "3306",
    "database.user": "debezium_user",
    "database.password": "${file:/secrets/mysql:password}",
    "database.server.id": "184054",
    "database.server.name": "production-mysql",

    "database.include.list": "ecommerce,inventory",
    "table.include.list":
    "ecommerce.orders,ecommerce.order_items,inventory.products",

    "database.history.kafka.bootstrap.servers": "kafka:9092",
    "database.history.kafka.topic": "schema-changes-mysql",

    "snapshot.mode": "initial",
    "snapshot.locking.mode": "minimal",

    "include.schema.changes": "true",
    "transforms": "unwrap,addMetadata",

    "transforms.unwrap.type": "io.debezium.transforms.ExtractNewRecordState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "rewrite",
    "transforms.unwrap.add.fields": "table,lsn,source.ts_ms",

    "transforms.addMetadata.type":
    "org.apache.kafka.connect.transforms.InsertField$Value",
    "transforms.addMetadata.static.field": "source_system",
    "transforms.addMetadata.static.value": "production-mysql",
  }
}

```

```

    "heartbeat.interval.ms": "10000",
    "heartbeat.topics.prefix": "__debezium-heartbeat",

    "decimal.handling.mode": "precise",
    "time.precision.mode": "adaptive_time_microseconds",

    "event.processing.failure.handling.mode": "warn",
    "inconsistent.schema.handling.mode": "warn"
  }
}

```

PostgreSQL CDC Configuration:

```

{
  "name": "postgres-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
    "plugin.name": "pgoutput",
    "slot.name": "debezium_slot",

    "database.hostname": "postgres.prod.company.com",
    "database.port": "5432",
    "database.user": "debezium_user",
    "database.password": "${file:/secrets/postgres:password}",
    "database.dbname": "production",
    "database.server.name": "production-postgres",

    "table.include.list": "public.users,public.transactions",

    "publication.name": "debezium_publication",
    "publication.autocreate.mode": "filtered",

    "snapshot.mode": "initial",

    "schema.history.internal.kafka.bootstrap.servers": "kafka:9092",
    "schema.history.internal.kafka.topic": "schema-changes-postgres",

    "heartbeat.interval.ms": "10000",
    "heartbeat.action.query": "INSERT INTO heartbeat (ts) VALUES (NOW())",

    "transforms": "route",
    "transforms.route.type": "org.apache.kafka.connect.transforms.RegexRouter",
    "transforms.route.regex": "([^.]+)\\.([^.]+)\\.([^.]+)",
    "transforms.route.replacement": "cdc.$3"
  }
}

```

MongoDB CDC Configuration:

```
{
  "name": "mongodb-cdc-connector",
  "config": {
    "connector.class": "io.debezium.connector.mongodb.MongoDbConnector",
    "mongodb.hosts": "rs0/mongo1:27017,mongo2:27017,mongo3:27017",
    "mongodb.name": "production-mongodb",
    "mongodb.user": "debezium_user",
    "mongodb.password": "${file:/secrets/mongodb:password}",

    "database.include.list": "ecommerce,users",
    "collection.include.list": "ecommerce.orders,users.profiles",

    "snapshot.mode": "initial",

    "transforms": "unwrap",
    "transforms.unwrap.type":
"io.debezium.connector.mongodb.transforms.ExtractNewDocumentState",
    "transforms.unwrap.drop.tombstones": "false",
    "transforms.unwrap.delete.handling.mode": "drop",
    "transforms.unwrap.add.fields": "op,source.ts_ms"
  }
}
```

CDC Event Structure

MySQL Insert Event:

```
{
  "before": null,
  "after": {
    "order_id": 12345,
    "user_id": 789,
    "amount": 99.99,
    "status": "pending",
    "created_at": 1705315800000,
    "updated_at": 1705315800000
  },
  "source": {
    "version": "2.5.0.Final",
    "connector": "mysql",
    "name": "production-mysql",
    "ts_ms": 1705315800123,
    "snapshot": "false",
    "db": "ecommerce",
    "sequence": null,
    "table": "orders",
    "server_id": 184054,
    "gtid": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa:12345",
    "file": "mysql-bin.000042",
    "pos": 154890,
    "row": 0,

```

```

    "thread": 127,
    "query": null
  },
  "op": "c",
  "ts_ms": 1705315800456,
  "transaction": null
}

```

MySQL Update Event:

```

{
  "before": {
    "order_id": 12345,
    "user_id": 789,
    "amount": 99.99,
    "status": "pending",
    "created_at": 1705315800000,
    "updated_at": 1705315800000
  },
  "after": {
    "order_id": 12345,
    "user_id": 789,
    "amount": 99.99,
    "status": "completed",
    "created_at": 1705315800000,
    "updated_at": 1705316900000
  },
  "source": {
    "version": "2.5.0.Final",
    "connector": "mysql",
    "name": "production-mysql",
    "ts_ms": 1705316900123,
    "snapshot": "false",
    "db": "ecommerce",
    "table": "orders",
    "server_id": 184054,
    "gtid": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa:12346",
    "file": "mysql-bin.000042",
    "pos": 155234,
    "row": 0,
    "thread": 128,
    "query": null
  },
  "op": "u",
  "ts_ms": 1705316900456,
  "transaction": null
}

```

Delete Event:

```
{
  "before": {
    "order_id": 12345,
    "user_id": 789,
    "amount": 99.99,
    "status": "completed",
    "created_at": 1705315800000,
    "updated_at": 1705316900000
  },
  "after": null,
  "source": {
    "version": "2.5.0.Final",
    "connector": "mysql",
    "name": "production-mysql",
    "ts_ms": 1705318000123,
    "snapshot": "false",
    "db": "ecommerce",
    "table": "orders",
    "server_id": 184054,
    "gtid": "aaaaaaaa-aaaa-aaaa-aaaa-aaaaaaaaaaaa:12347",
    "file": "mysql-bin.000042",
    "pos": 155678,
    "row": 0,
    "thread": 129,
    "query": null
  },
  "op": "d",
  "ts_ms": 1705318000456,
  "transaction": null
}
```

Processing CDC Events

Flink CDC Consumer:

```
import org.apache.flink.api.common.eventtime.WatermarkStrategy;
import org.apache.flink.connector.kafka.source.KafkaSource;
import
org.apache.flink.connector.kafka.source.enumerator.initializer.OffsetsInitializer;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;

public class CDCProcessor {

    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        // Enable checkpointing for exactly-once processing
```

```

    env.enableCheckpointing(60000); // 60 seconds

    // Kafka source for CDC events
    KafkaSource<String> kafkaSource = KafkaSource.<String>builder()
        .setBootstrapServers("kafka:9092")
        .setTopics("production-mysql.ecommerce.orders")
        .setGroupId("cdc-processor-group")
        .setStartingOffsets(OffsetsInitializer.earliest())
        .setValueOnlyDeserializer(new SimpleStringSchema())
        .build();

    DataStream<String> cdcEvents = env
        .fromSource(kafkaSource, WatermarkStrategy.noWatermarks(), "Kafka CDC
Source");

    // Parse and process CDC events
    DataStream<Order> orders = cdcEvents
        .map(new CDCEventParser())
        .filter(order -> order != null); // Filter out deletes if needed

    // Branch processing based on operation type
    DataStream<Order> inserts = orders.filter(o ->
o.operation.equals("INSERT"));
    DataStream<Order> updates = orders.filter(o ->
o.operation.equals("UPDATE"));
    DataStream<Order> deletes = orders.filter(o ->
o.operation.equals("DELETE"));

    // Process inserts: Write to data lake
    inserts.addSink(new S3Sink<> (
        "s3://data-lake/raw/orders/",
        new ParquetWriter<>()
    ));

    // Process updates: Update cache
    updates.addSink(new RedisSink<> (
        "order:",
        order -> order.orderId,
        order -> serializeOrder(order)
    ));

    // Process deletes: Mark as deleted (soft delete)
    deletes.addSink(new ElasticsearchSink<> (
        "orders-index",
        order -> createDeleteDocument(order)
    ));

    env.execute("CDC Processing Pipeline");
}

// Parse CDC event
public static class CDCEventParser implements MapFunction<String, Order> {
    private final ObjectMapper mapper = new ObjectMapper();

```

```

@Override
public Order map(String json) throws Exception {
    JsonNode root = mapper.readTree(json);

    String operation = root.get("op").asText();
    JsonNode after = root.get("after");
    JsonNode before = root.get("before");

    // For INSERT and UPDATE, use "after" state
    // For DELETE, use "before" state
    JsonNode data = operation.equals("d") ? before : after;

    if (data == null) return null;

    return new Order(
        data.get("order_id").asLong(),
        data.get("user_id").asLong(),
        data.get("amount").asDouble(),
        data.get("status").asText(),
        operation.equals("c") ? "INSERT" :
        operation.equals("u") ? "UPDATE" : "DELETE",
        root.get("ts_ms").asLong()
    );
}

// Order POJO
public static class Order {
    public long orderId;
    public long userId;
    public double amount;
    public String status;
    public String operation;
    public long timestamp;

    public Order(long orderId, long userId, double amount,
        String status, String operation, long timestamp) {
        this.orderId = orderId;
        this.userId = userId;
        this.amount = amount;
        this.status = status;
        this.operation = operation;
        this.timestamp = timestamp;
    }
}

```

Python CDC Consumer (with Kafka):

```

from kafka import KafkaConsumer
import json
import logging

```

```

from typing import Dict, Optional
from dataclasses import dataclass
from datetime import datetime

@dataclass
class CDCEvent:
    operation: str # INSERT, UPDATE, DELETE
    table: str
    before: Optional[Dict]
    after: Optional[Dict]
    source_timestamp: int
    processing_timestamp: int

class CDCEventProcessor:
    """
    Process CDC events from Kafka
    """

    def __init__(self, bootstrap_servers: str, group_id: str):
        self.consumer = KafkaConsumer(
            bootstrap_servers=bootstrap_servers,
            group_id=group_id,
            value_deserializer=lambda m: json.loads(m.decode('utf-8')),
            auto_offset_reset='earliest',
            enable_auto_commit=True,
            auto_commit_interval_ms=5000
        )
        self.logger = logging.getLogger(__name__)

    def subscribe_topics(self, topics: list):
        """Subscribe to CDC topics"""
        self.consumer.subscribe(topics)
        self.logger.info(f"Subscribed to topics: {topics}")

    def parse_cdc_event(self, message: dict) -> CDCEvent:
        """Parse Debezium CDC event"""
        operation_map = {
            'c': 'INSERT',
            'u': 'UPDATE',
            'd': 'DELETE',
            'r': 'READ' # Initial snapshot
        }

        operation = operation_map.get(message['op'], 'UNKNOWN')

        return CDCEvent(
            operation=operation,
            table=message['source']['table'],
            before=message.get('before'),
            after=message.get('after'),
            source_timestamp=message.get('ts_ms'),
            processing_timestamp=int(datetime.now().timestamp() * 1000)
        )

```

```
def process_insert(self, event: CDCEvent):
    """Handle INSERT operations"""
    data = event.after
    self.logger.info(f"INSERT into {event.table}: {data}")

    # Write to data lake
    self.write_to_data_lake(event.table, data, 'insert')

    # Update cache
    self.update_cache(event.table, data['id'], data)

    # Send to downstream systems
    self.publish_to_downstream(event.table, 'created', data)

def process_update(self, event: CDCEvent):
    """Handle UPDATE operations"""
    before = event.before
    after = event.after

    self.logger.info(f"UPDATE {event.table}: {before} -> {after}")

    # Detect which fields changed
    changed_fields = {
        k: {'old': before[k], 'new': after[k]}
        for k in after.keys()
        if before.get(k) != after.get(k)
    }

    self.logger.info(f"Changed fields: {changed_fields}")

    # Write to data lake
    self.write_to_data_lake(event.table, after, 'update')

    # Update cache
    self.update_cache(event.table, after['id'], after)

    # Send change event to downstream
    self.publish_to_downstream(event.table, 'updated', {
        'current': after,
        'previous': before,
        'changes': changed_fields
    })

def process_delete(self, event: CDCEvent):
    """Handle DELETE operations"""
    data = event.before
    self.logger.info(f"DELETE from {event.table}: {data}")

    # Mark as deleted in data lake (soft delete)
    self.write_to_data_lake(event.table, {
        **data,
        'deleted_at': event.processing_timestamp,
        'is_deleted': True
    }, 'delete')
```

```

    # Remove from cache
    self.remove_from_cache(event.table, data['id'])

    # Send delete event to downstream
    self.publish_to_downstream(event.table, 'deleted', data)

def run(self):
    """Main processing loop"""
    self.logger.info("Starting CDC event processor...")

    for message in self.consumer:
        try:
            # Parse CDC event
            cdc_event = self.parse_cdc_event(message.value)

            # Route based on operation type
            if cdc_event.operation == 'INSERT':
                self.process_insert(cdc_event)
            elif cdc_event.operation == 'UPDATE':
                self.process_update(cdc_event)
            elif cdc_event.operation == 'DELETE':
                self.process_delete(cdc_event)
            else:
                self.logger.warning(f"Unknown operation:
{cdc_event.operation}")

        except Exception as e:
            self.logger.error(f"Error processing message: {e}", exc_info=True)
            # Implement error handling: DLQ, retry, etc.

def write_to_data_lake(self, table: str, data: dict, operation: str):
    """Write to data lake (S3/ADLS/GCS)"""
    # Implementation: write as Parquet to S3
    pass

def update_cache(self, table: str, key: str, data: dict):
    """Update cache (Redis/Memcached)"""
    # Implementation: SET key value in Redis
    pass

def remove_from_cache(self, table: str, key: str):
    """Remove from cache"""
    # Implementation: DEL key from Redis
    pass

def publish_to_downstream(self, table: str, event_type: str, data: dict):
    """Publish to downstream systems"""
    # Implementation: publish to another Kafka topic, webhook, etc.
    pass

# Usage
if __name__ == "__main__":
    logging.basicConfig(level=logging.INFO)

```

```

processor = CDCEventProcessor(
    bootstrap_servers='kafka:9092',
    group_id='cdc-processor-group'
)

processor.subscribe_topics([
    'production-mysql.ecommerce.orders',
    'production-mysql.ecommerce.order_items',
    'production-postgres.public.users'
])

processor.run()

```

Schema Evolution Handling

Scenario: New column added to table

```

-- Database: Add new column
ALTER TABLE orders ADD COLUMN payment_method VARCHAR(50);

-- Debezium automatically captures schema change
-- New events will include payment_method field

```

Flink Schema Evolution Handler:

```

public class SchemaEvolutionHandler {

    public static void handleSchemaChange(JsonNode event) {
        JsonNode schema = event.get("schema");

        if (schema != null) {
            // Schema change detected
            String table = event.get("source").get("table").asText();

            // Register new schema version
            SchemaRegistry.registerNewVersion(table, schema);

            // Update downstream systems
            notifySchemaChange(table, schema);
        }
    }

    public static Order parseOrderWithSchemaVersion(JsonNode data, int
schemaVersion) {
        Order order = new Order();

        // Always present fields
        order.orderId = data.get("order_id").asLong();
    }
}

```

```

        order.amount = data.get("amount").asDouble();

        // Handle optional fields based on schema version
        if (schemaVersion >= 2 && data.has("payment_method")) {
            order.paymentMethod = data.get("payment_method").asText();
        } else {
            order.paymentMethod = "unknown"; // Default value
        }

        return order;
    }
}

```

CDC Monitoring & Alerting

Key Metrics to Monitor:

```

from prometheus_client import Counter, Gauge, Histogram
import time

# Metrics
cdc_events_processed = Counter(
    'cdc_events_processed_total',
    'Total CDC events processed',
    ['table', 'operation']
)

cdc_lag_seconds = Gauge(
    'cdc_lag_seconds',
    'Lag between source timestamp and processing time',
    ['table']
)

cdc_processing_duration = Histogram(
    'cdc_processing_duration_seconds',
    'Time to process CDC event',
    ['table']
)

def process_with_metrics(event: CDCEvent):
    """Process event with metrics collection"""
    start_time = time.time()

    try:
        # Process event
        if event.operation == 'INSERT':
            process_insert(event)
        # ... other operations

        # Record success metrics
        cdc_events_processed.labels(

```

```

        table=event.table,
        operation=event.operation
    ).inc()

    # Calculate lag
    lag = (event.processing_timestamp - event.source_timestamp) / 1000.0
    cdc_lag_seconds.labels(table=event.table).set(lag)

finally:
    # Record processing duration
    duration = time.time() - start_time
    cdc_processing_duration.labels(table=event.table).observe(duration)

```

Alert Rules:

```

# Prometheus alert rules
groups:
- name: cdc_alerts
  rules:
  - alert: CDCLagHigh
    expr: cdc_lag_seconds > 300
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "CDC lag is high for table {{ $labels.table }}"
      description: "Lag is {{ $value }} seconds"

  - alert: CDCEventsStalled
    expr: rate(cdc_events_processed_total[5m]) == 0
    for: 10m
    labels:
      severity: critical
    annotations:
      summary: "No CDC events processed for table {{ $labels.table }}"
      description: "Event processing appears to be stalled"

  - alert: CDCProcessingSlowdown
    expr: rate(cdc_processing_duration_seconds_sum[5m]) /
rate(cdc_processing_duration_seconds_count[5m]) > 1
    for: 5m
    labels:
      severity: warning
    annotations:
      summary: "CDC processing is slow for table {{ $labels.table }}"
      description: "Average processing time is {{ $value }} seconds"

```

CDC Best Practices

1. Database Configuration:

```
-- MySQL: Enable binlog
[mysqld]
server-id = 1
log_bin = mysql-bin
binlog_format = ROW
binlog_row_image = FULL
expire_logs_days = 7

-- PostgreSQL: Enable logical replication
wal_level = logical
max_wal_senders = 10
max_replication_slots = 10
```

2. Monitoring Binlog Position:

```
def check_binlog_lag():
    """Check if Debezium is keeping up with binlog"""
    # Get current binlog position from MySQL
    current_binlog = get_mysql_binlog_position()

    # Get Debezium's binlog position from offset storage
    debezium_binlog = get_debezium_binlog_position()

    # Calculate lag
    lag_bytes = current_binlog['position'] - debezium_binlog['position']

    if lag_bytes > 100_000_000: # 100 MB lag
        alert("High binlog lag detected")
```

3. Handling Initial Snapshot:

```
{
  "snapshot.mode": "initial",
  "snapshot.fetch.size": "10000",
  "snapshot.max.threads": "4",
  "snapshot.delay.ms": "0",

  "min.row.count.to.stream.results": "1000"
}
```

4. Error Handling:

```
// Dead Letter Queue for failed events
public class CDCErrorHandler {
    private final KafkaProducer<String, String> dlqProducer;
```

```

    public void handleProcessingError(CDCEvent event, Exception error) {
        // Log error
        logger.error("Failed to process CDC event", error);

        // Send to DLQ
        ProducerRecord<String, String> dlqRecord = new ProducerRecord<>(
            "cdc-errors",
            event.table,
            createErrorPayload(event, error)
        );

        dlqProducer.send(dlqRecord);

        // Update metrics
        errorCounter.labels(event.table, error.getClass().getName()).inc();
    }

    private String createErrorPayload(CDCEvent event, Exception error) {
        return JsonBuilder.create()
            .add("event", event.toJson())
            .add("error", error.getMessage())
            .add("timestamp", System.currentTimeMillis())
            .add("retry_count", getRetryCount(event))
            .build();
    }
}

```

When to Use CDC

☑ Use CDC When:

- Need real-time data synchronization
- Want to maintain event history
- Multiple systems need same data
- Database can't handle additional query load
- Need to track all changes for audit/compliance

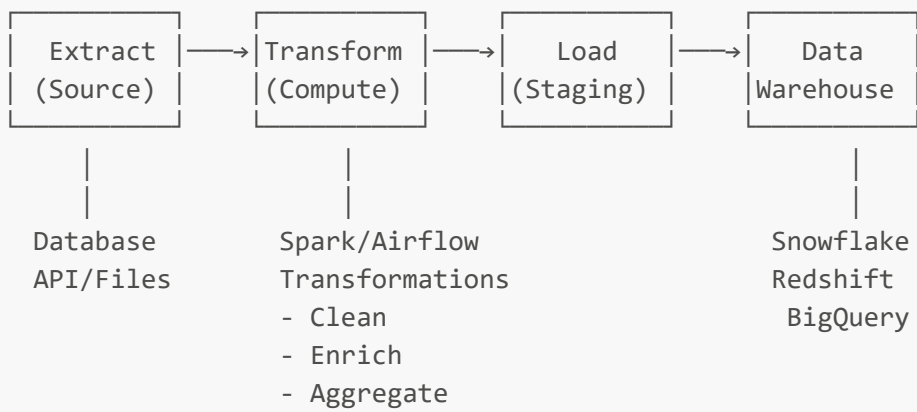
✗ Don't Use CDC When:

- Batch updates are sufficient (use ETL)
- Can't enable binlog/WAL (use API polling)
- Data volume is extremely high (> 1M changes/second)
- Source database doesn't support CDC

3. ETL vs ELT Patterns {#etl-elt}

ETL (Extract, Transform, Load)

Traditional approach: Transform data before loading into warehouse



ETL Example with Apache Spark:

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

class ETLPipeline:
    """
    Traditional ETL pipeline with Spark
    """

    def __init__(self):
        self.spark = SparkSession.builder \
            .appName("ETL Pipeline") \
            .config("spark.sql.adaptive.enabled", "true") \
            .getOrCreate()

    def extract(self):
        """Extract data from sources"""
        # Extract from PostgreSQL
        orders = self.spark.read \
            .format("jdbc") \
            .option("url", "jdbc:postgresql://db:5432/ecommerce") \
            .option("dbtable", "orders") \
            .option("user", "user") \
            .option("password", "password") \
            .load()

        # Extract from S3 (customer data)
        customers = self.spark.read \
            .parquet("s3://raw-data/customers/")

        # Extract from API (enrichment data)
        products = self.spark.read \
            .json("s3://raw-data/products/")

        return orders, customers, products

    def transform(self, orders, customers, products):

```

```

"""Transform data"""
# 1. Data Quality: Remove nulls, fix data types
orders_clean = orders \
    .filter(col("order_id").isNotNull()) \
    .withColumn("amount", col("amount").cast("decimal(10,2)")) \
    .withColumn("order_date", to_timestamp("order_date"))

# 2. Enrich: Join with customer and product data
enriched = orders_clean \
    .join(customers, "customer_id", "left") \
    .join(products, "product_id", "left")

# 3. Business Logic: Calculate metrics
transformed = enriched \
    .withColumn("revenue", col("amount") * col("quantity")) \
    .withColumn("profit_margin",
        (col("amount") - col("cost")) / col("amount")) \
    .withColumn("customer_segment",
        when(col("total_spend") > 10000, "VIP")
        .when(col("total_spend") > 1000, "Premium")
        .otherwise("Standard"))

# 4. Aggregate: Create summary tables
daily_revenue = transformed \
    .groupBy(
        window("order_date", "1 day"),
        "product_category"
    ) \
    .agg(
        sum("revenue").alias("total_revenue"),
        count("*").alias("order_count"),
        avg("profit_margin").alias("avg_margin")
    )

return transformed, daily_revenue

def load(self, transformed, daily_revenue):
    """Load into data warehouse"""
    # Load detailed transactions
    transformed.write \
        .mode("overwrite") \
        .partitionBy("order_date") \
        .parquet("s3://warehouse/transactions/")

    # Load to Snowflake
    transformed.write \
        .format("snowflake") \
        .options(**snowflake_options) \
        .option("dbtable", "analytics.orders") \
        .mode("append") \
        .save()

    # Load aggregates
    daily_revenue.write \

```

```

        .format("snowflake") \
        .options(**snowflake_options) \
        .option("dbtable", "analytics.daily_revenue") \
        .mode("overwrite") \
        .save()

def run(self):
    """Execute ETL pipeline"""
    # Extract
    orders, customers, products = self.extract()

    # Transform
    transformed, daily_revenue = self.transform(orders, customers, products)

    # Load
    self.load(transformed, daily_revenue)

    self.spark.stop()

# Airflow DAG for scheduling
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta

default_args = {
    'owner': 'data-eng',
    'depends_on_past': False,
    'email_on_failure': True,
    'email': ['data-eng@company.com'],
    'retries': 3,
    'retry_delay': timedelta(minutes=5),
}

dag = DAG(
    'etl_pipeline',
    default_args=default_args,
    description='Daily ETL pipeline',
    schedule_interval='0 2 * * *', # 2 AM daily
    start_date=datetime(2024, 1, 1),
    catchup=False,
)

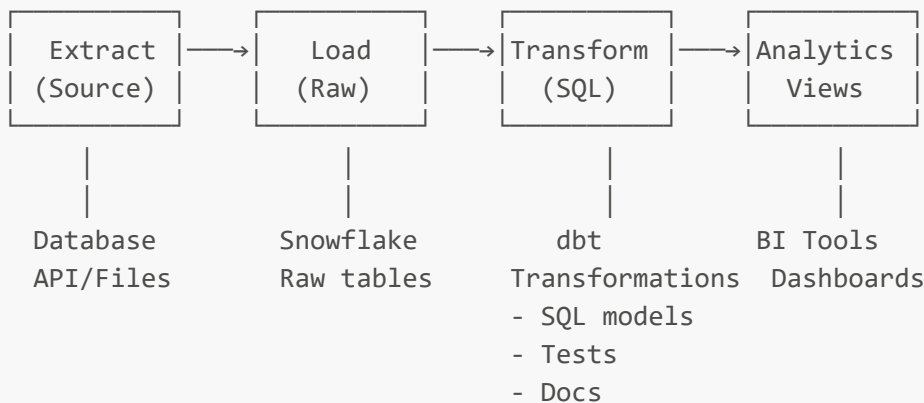
def run_etl():
    pipeline = ETLPipeline()
    pipeline.run()

etl_task = PythonOperator(
    task_id='run_etl_pipeline',
    python_callable=run_etl,
    dag=dag,
)

```

ELT (Extract, Load, Transform)

Modern approach: Load raw data first, transform in warehouse



ELT Example with Fivetran + dbt:

Step 1: Load with Fivetran (Configuration)

```

{
  "connector": "postgres",
  "config": {
    "host": "postgres.prod.company.com",
    "port": 5432,
    "database": "ecommerce",
    "user": "fivetran_user",
    "password": "${POSTGRES_PASSWORD}",
    "schemas": ["public"],
    "sync_mode": "INCREMENTAL",
    "update_method": "XMIN"
  },
  "destination": {
    "type": "snowflake",
    "schema": "raw_postgres"
  },
  "schedule": {
    "sync_frequency": "15",
    "sync_frequency_unit": "minutes"
  }
}

```

Step 2: Transform with dbt

```

-- models/staging/stg_orders.sql
{{ config(
  materialized='view',
  tags=['staging']
)

```

```

) }}

WITH source AS (
  SELECT * FROM {{ source('raw_postgres', 'orders') }}
),

cleaned AS (
  SELECT
    order_id,
    customer_id,
    product_id,
    CAST(amount AS DECIMAL(10,2)) AS amount,
    CAST(quantity AS INTEGER) AS quantity,
    TO_TIMESTAMP(order_date) AS order_date,
    status,
    _fivetran_synced AS ingestion_timestamp
  FROM source
  WHERE order_id IS NOT NULL
    AND amount > 0
    AND order_date >= DATEADD(year, -2, CURRENT_DATE())
)

SELECT * FROM cleaned

-- models/marts/fct_orders.sql
{{ config(
  materialized='incremental',
  unique_key='order_id',
  tags=['marts', 'orders']
)}}

WITH orders AS (
  SELECT * FROM {{ ref('stg_orders') }}
),

customers AS (
  SELECT * FROM {{ ref('stg_customers') }}
),

products AS (
  SELECT * FROM {{ ref('stg_products') }}
),

enriched AS (
  SELECT
    o.order_id,
    o.customer_id,
    o.product_id,
    o.amount,
    o.quantity,
    o.order_date,
    o.status,

    -- Customer attributes

```

```

        c.customer_name,
        c.customer_segment,
        c.lifetime_value,

        -- Product attributes
        p.product_name,
        p.product_category,
        p.unit_cost,

        -- Calculated metrics
        o.amount * o.quantity AS revenue,
        (o.amount - p.unit_cost) * o.quantity AS profit,
        (o.amount - p.unit_cost) / o.amount AS profit_margin,

        -- Metadata
        o.ingestion_timestamp,
        CURRENT_TIMESTAMP() AS transformation_timestamp

FROM orders o
LEFT JOIN customers c ON o.customer_id = c.customer_id
LEFT JOIN products p ON o.product_id = p.product_id

{% if is_incremental() %}
    WHERE o.order_date > (SELECT MAX(order_date) FROM {{ this }})
{% endif %}
)

SELECT * FROM enriched

-- models/marts/agg_daily_revenue.sql
{{ config(
    materialized='table',
    tags=['marts', 'aggregates']
)}}

SELECT
    DATE_TRUNC('day', order_date) AS date,
    product_category,
    customer_segment,

    -- Revenue metrics
    SUM(revenue) AS total_revenue,
    SUM(profit) AS total_profit,
    AVG(profit_margin) AS avg_profit_margin,

    -- Volume metrics
    COUNT(DISTINCT order_id) AS order_count,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(quantity) AS total_quantity,

    -- Performance metrics
    AVG(revenue) AS avg_order_value,
    SUM(revenue) / COUNT(DISTINCT customer_id) AS revenue_per_customer

```

```
FROM {{ ref('fct_orders') }}
WHERE status = 'completed'
GROUP BY 1, 2, 3
```

dbt Tests (Data Quality):

```
# models/schema.yml
version: 2

models:
  - name: fct_orders
    description: "Fact table for orders"
    columns:
      - name: order_id
        description: "Unique order identifier"
        tests:
          - unique
          - not_null

      - name: amount
        description: "Order amount"
        tests:
          - not_null
          - dbt_utils.accepted_range:
              min_value: 0
              max_value: 1000000

      - name: order_date
        description: "Order timestamp"
        tests:
          - not_null
          - dbt_utils.recency:
              datepart: day
              interval: 1

      - name: customer_id
        description: "Customer identifier"
        tests:
          - not_null
          - relationships:
              to: ref('stg_customers')
              field: customer_id
```

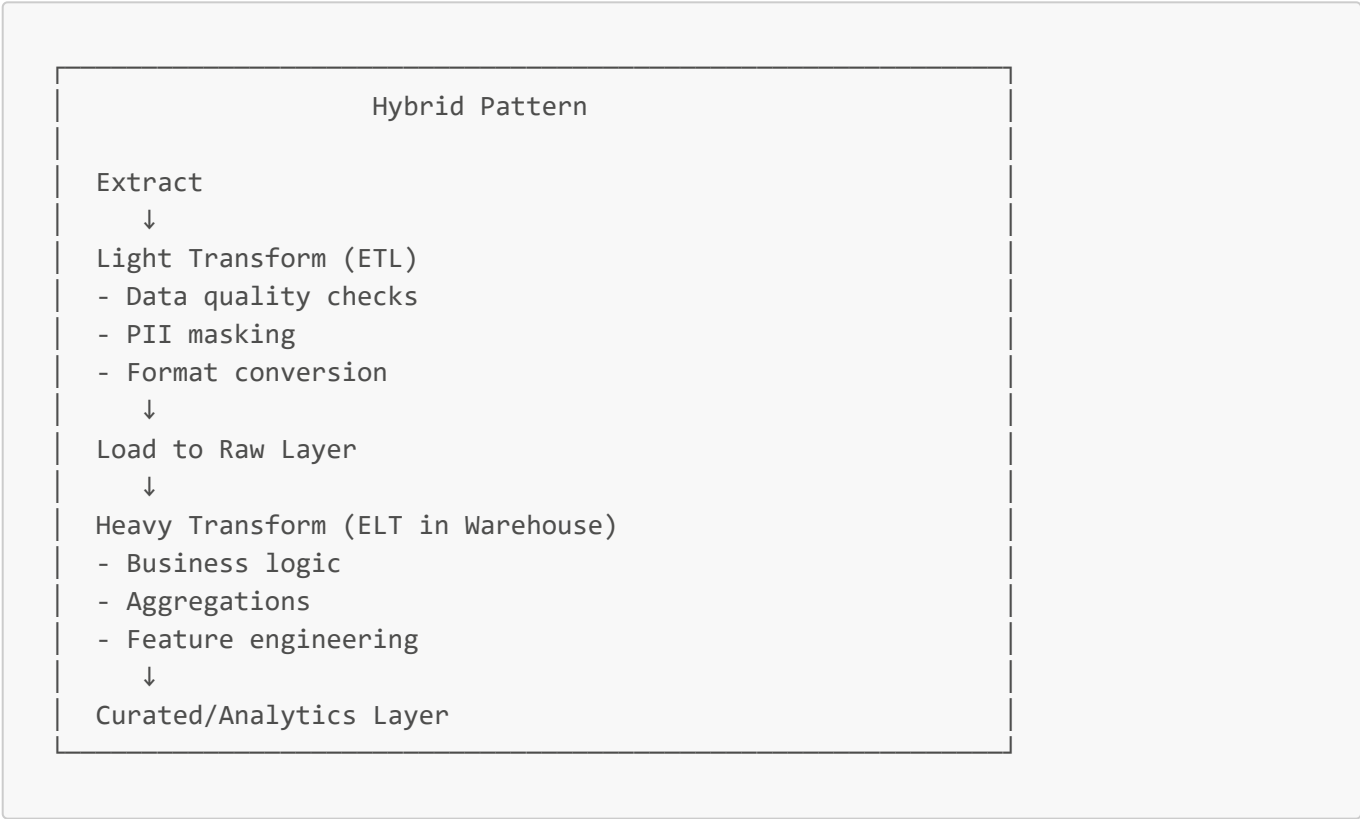
ETL vs ELT Comparison

Aspect	ETL	ELT
Transform Location	External compute (Spark, Airflow)	In warehouse (SQL)
Data Model	Pre-defined schema	Schema-on-read

Aspect	ETL	ELT
Infrastructure	Separate compute cluster	Warehouse compute
Flexibility	Less flexible (re-run pipeline)	More flexible (re-query)
Cost	Compute + warehouse	Warehouse compute only
Latency	Higher (two-step)	Lower (one load)
Best For	Complex transformations, ML prep	BI, reporting, ad-hoc analysis
Skillset	Python, Spark, Java	SQL, dbt
Data Quality	Pre-load validation	Post-load testing

Hybrid ETL/ELT Pattern

Best of Both Worlds:



```
# Hybrid pipeline
def hybrid_pipeline():
    # Step 1: Light ETL (data quality, PII masking)
    raw_data = extract_from_source()

    cleaned = raw_data \
        .filter(col("id").isNotNull()) \
        .withColumn("email", mask_pii(col("email"))) \
        .withColumn("phone", mask_pii(col("phone")))

    # Step 2: Load to raw layer
```

```
cleaned.write.parquet("s3://lake/raw/orders/")

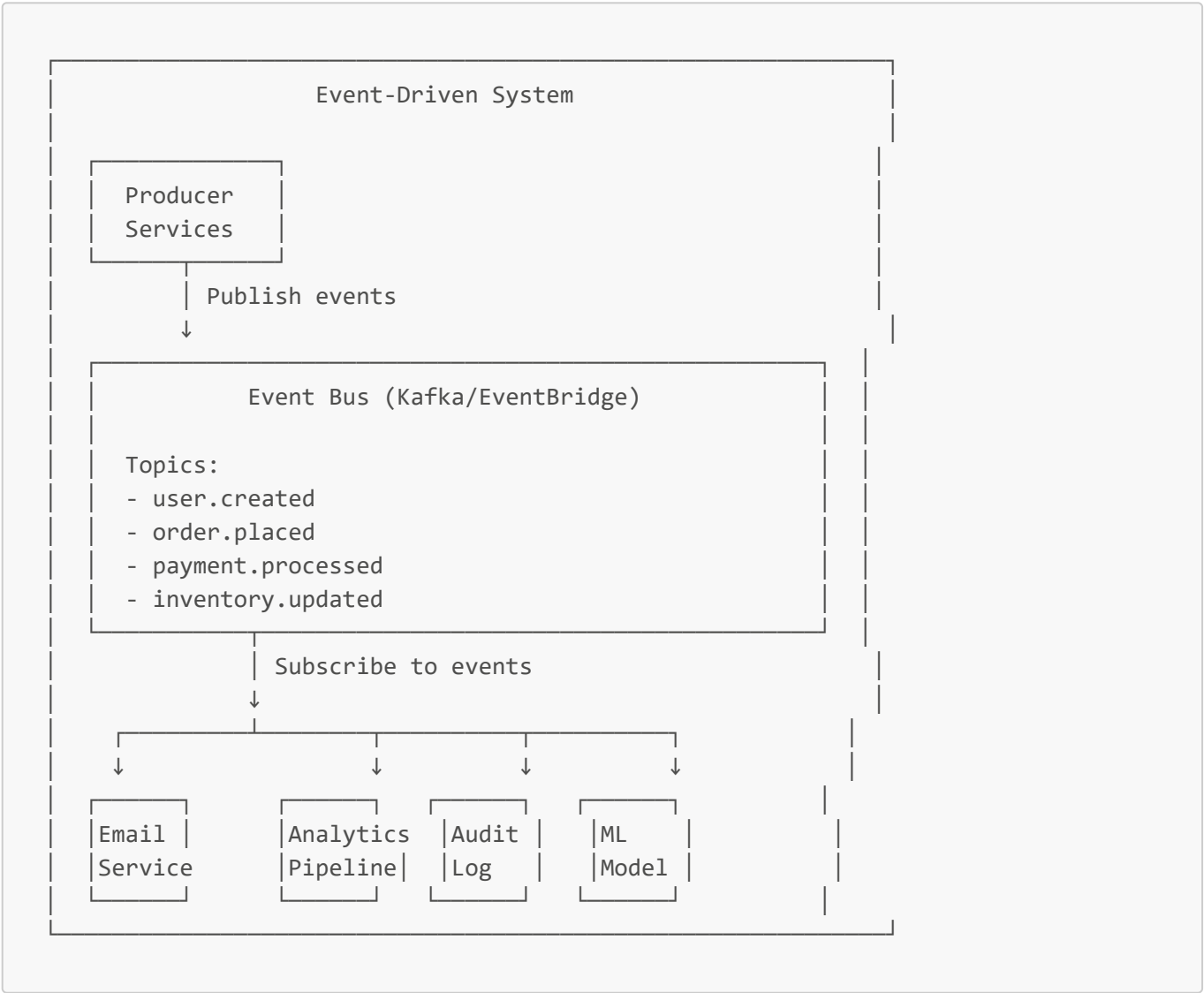
# Step 3: Heavy ELT in warehouse (dbt)
# Warehouse performs complex joins, aggregations, feature engineering
# Using dbt models
```

4. Event-Driven Architecture {#event-driven}

Overview

Event-driven architecture uses **events** as the primary mechanism for communication between services. When something happens (an event), interested parties are notified and can react.

Architecture



Event Schema Design

CloudEvents Standard:

```
{
  "specversion": "1.0",
  "type": "com.company.order.placed",
  "source": "https://api.company.com/orders",
  "id": "A234-1234-1234",
  "time": "2026-08-15T10:30:00Z",
  "datacontenttype": "application/json",
  "data": {
    "order_id": "ORD-12345",
    "customer_id": "CUST-789",
    "amount": 99.99,
    "currency": "USD",
    "items": [
      {
        "product_id": "PROD-111",
        "quantity": 2,
        "price": 49.995
      }
    ],
    "shipping_address": {
      "street": "123 Main St",
      "city": "San Francisco",
      "state": "CA",
      "zip": "94102"
    },
    "payment_method": "credit_card"
  }
}
```

Producer Implementation

Event Publisher (Python):

```
from kafka import KafkaProducer
from dataclasses import dataclass, asdict
from typing import Dict, Any
import json
import uuid
from datetime import datetime

@dataclass
class Event:
    """Base event class"""
    event_type: str
    event_id: str
    timestamp: str
    source: str
    data: Dict[str, Any]

    def to_cloudevents(self) -> dict:
        """Convert to CloudEvents format"""
```

```

        return {
            "specversion": "1.0",
            "type": self.event_type,
            "source": self.source,
            "id": self.event_id,
            "time": self.timestamp,
            "datacontenttype": "application/json",
            "data": self.data
        }

class EventPublisher:
    """
    Publish events to Kafka with retry and monitoring
    """

    def __init__(self, bootstrap_servers: str):
        self.producer = KafkaProducer(
            bootstrap_servers=bootstrap_servers,
            value_serializer=lambda v: json.dumps(v).encode('utf-8'),
            acks='all', # Wait for all replicas
            retries=3,
            max_in_flight_requests_per_connection=5,
            enable_idempotence=True, # Exactly-once
            compression_type='snappy'
        )

    def publish(self, topic: str, event_type: str,
                data: dict, key: str = None):
        """
        Publish event with automatic metadata
        """
        event = Event(
            event_type=event_type,
            event_id=str(uuid.uuid4()),
            timestamp=datetime.utcnow().isoformat() + 'Z',
            source="order-service",
            data=data
        )

        cloudevents_payload = event.to_cloudevents()

        try:
            # Send to Kafka
            future = self.producer.send(
                topic,
                key=key.encode('utf-8') if key else None,
                value=cloudevents_payload
            )

            # Wait for acknowledgment
            record_metadata = future.get(timeout=10)

            print(f"Event published: {event.event_id}")
            print(f"Topic: {record_metadata.topic}")

```

```

        print(f"Partition: {record_metadata.partition}")
        print(f"Offset: {record_metadata.offset}")

        return event.event_id

    except Exception as e:
        print(f"Failed to publish event: {e}")
        raise

def close(self):
    """Flush and close producer"""
    self.producer.flush()
    self.producer.close()

# Usage in application
class OrderService:
    """
    Order service that publishes events
    """

    def __init__(self):
        self.event_publisher = EventPublisher('kafka:9092')

    def create_order(self, customer_id: str, items: list) -> dict:
        """Create order and publish event"""
        # Business logic
        order = {
            'order_id': generate_order_id(),
            'customer_id': customer_id,
            'items': items,
            'amount': calculate_total(items),
            'status': 'pending',
            'created_at': datetime.utcnow().isoformat()
        }

        # Save to database
        save_order_to_db(order)

        # Publish event
        self.event_publisher.publish(
            topic='order.events',
            event_type='com.company.order.created',
            data=order,
            key=order['order_id']
        )

        return order

    def update_order_status(self, order_id: str, new_status: str):
        """Update order status and publish event"""
        # Update database
        update_order_in_db(order_id, {'status': new_status})

        # Publish event

```

```

        self.event_publisher.publish(
            topic='order.events',
            event_type='com.company.order.status_changed',
            data={
                'order_id': order_id,
                'old_status': get_old_status(order_id),
                'new_status': new_status,
                'changed_at': datetime.utcnow().isoformat()
            },
            key=order_id
        )

```

Consumer Implementation

Event Consumer (Flink):

```

import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.flink.streaming.api.environment.StreamExecutionEnvironment;
import org.apache.flink.connector.kafka.source.KafkaSource;
import org.apache.flink.api.common.serialization.SimpleStringSchema;

public class EventConsumerPipeline {

    public static void main(String[] args) throws Exception {
        StreamExecutionEnvironment env =
            StreamExecutionEnvironment.getExecutionEnvironment();

        // Kafka source for events
        KafkaSource<String> source = KafkaSource.<String>builder()
            .setBootstrapServers("kafka:9092")
            .setTopics("order.events", "payment.events")
            .setGroupId("analytics-consumer")
            .setStartingOffsets(OffsetsInitializer.earliest())
            .setValueOnlyDeserializer(new SimpleStringSchema())
            .build();

        DataStream<String> events = env.fromSource(
            source,
            WatermarkStrategy.nowatermarks(),
            "Kafka Source"
        );

        // Parse CloudEvents
        DataStream<CloudEvent> cloudEvents = events
            .map(new CloudEventParser());

        // Route events based on type
        DataStream<CloudEvent> orderEvents = cloudEvents
            .filter(event -> event.getType().startsWith("com.company.order."));

        DataStream<CloudEvent> paymentEvents = cloudEvents

```

```

        .filter(event -> event.getType().startsWith("com.company.payment."));

// Process order events
orderEvents
    .keyBy(event -> event.getData().get("order_id"))
    .process(new OrderEventProcessor())
    .addSink(new DataLakeSink("s3://lake/order-events/"));

// Process payment events
paymentEvents
    .keyBy(event -> event.getData().get("payment_id"))
    .process(new PaymentEventProcessor())
    .addSink(new DataLakeSink("s3://lake/payment-events/"));

env.execute("Event Consumer Pipeline");
}

// Event processor
public static class OrderEventProcessor
    extends KeyedProcessFunction<String, CloudEvent, EnrichedOrder> {

    private ValueState<OrderState> orderState;

    @Override
    public void open(Configuration config) {
        ValueStateDescriptor<OrderState> descriptor =
            new ValueStateDescriptor<>("order-state", OrderState.class);
        orderState = getRuntimeContext().getState(descriptor);
    }

    @Override
    public void processElement(
        CloudEvent event,
        Context ctx,
        Collector<EnrichedOrder> out) throws Exception {

        // Get current state
        OrderState state = orderState.value();
        if (state == null) {
            state = new OrderState();
        }

        // Update state based on event type
        switch (event.getType()) {
            case "com.company.order.created":
                state.setOrderId(event.getData().get("order_id"));
                state.setAmount(event.getData().get("amount"));
                state.setStatus("created");
                state.setCreatedAt(event.getTime());
                break;

            case "com.company.order.paid":
                state.setStatus("paid");
                state.setPaidAt(event.getTime());

```

```

        break;

        case "com.company.order.shipped":
            state.setStatus("shipped");
            state.setShippedAt(event.getTime());
            break;

        case "com.company.order.delivered":
            state.setStatus("delivered");
            state.setDeliveredAt(event.getTime());

            // Calculate metrics
            long timeToDeliver = state.getDeliveredAt() -
state.getCreatedAt();
            state.setTimeToDeliver(timeToDeliver);
            break;
    }

    // Update state
    orderState.update(state);

    // Emit enriched order
    out.collect(new EnrichedOrder(state));
}
}
}

```

Event Choreography vs Orchestration

Choreography (Decentralized):

Order Created Event

↓

- ├→ Inventory Service (reduce stock)
- ├→ Payment Service (charge customer)
- ├→ Notification Service (send email)
- └→ Analytics Service (record metrics)

Each service independently decides what to do

Orchestration (Centralized):

Order Orchestrator

- ├ Step 1: Call Inventory Service
- ├ Step 2: Call Payment Service
- ├ Step 3: Call Notification Service
- └ Step 4: Call Analytics Service

Central coordinator manages flow

Saga Pattern Example (Orchestration):

```
from enum import Enum
from dataclasses import dataclass
from typing import List, Callable

class SagaStatus(Enum):
    PENDING = "pending"
    COMPLETED = "completed"
    COMPENSATING = "compensating"
    FAILED = "failed"

@dataclass
class SagaStep:
    name: str
    action: Callable
    compensation: Callable
    status: SagaStatus = SagaStatus.PENDING

class OrderSaga:
    """
    Saga for order processing with compensation
    """

    def __init__(self, order_id: str):
        self.order_id = order_id
        self.steps = []
        self.completed_steps = []

    def add_step(self, name: str, action: Callable, compensation: Callable):
        """Add step to saga"""
        self.steps.append(SagaStep(name, action, compensation))

    def execute(self):
        """Execute saga with automatic compensation on failure"""
        try:
            # Execute each step
            for step in self.steps:
                print(f"Executing step: {step.name}")
                step.action()
                step.status = SagaStatus.COMPLETED
                self.completed_steps.append(step)

            print("Saga completed successfully")
            return True

        except Exception as e:
            print(f"Saga failed at step {step.name}: {e}")
            self.compensate()
            return False
```

```

def compensate(self):
    """Compensate by rolling back completed steps"""
    print("Starting compensation...")

    # Compensate in reverse order
    for step in reversed(self.completed_steps):
        try:
            print(f"Compensating step: {step.name}")
            step.compensation()
            step.status = SagaStatus.COMPENSATING
        except Exception as e:
            print(f"Compensation failed for {step.name}: {e}")

# Usage
def create_order_saga(order):
    saga = OrderSaga(order['order_id'])

    # Step 1: Reserve inventory
    saga.add_step(
        name="reserve_inventory",
        action=lambda: inventory_service.reserve(order['items']),
        compensation=lambda: inventory_service.release(order['items'])
    )

    # Step 2: Process payment
    saga.add_step(
        name="process_payment",
        action=lambda: payment_service.charge(order['customer_id'],
order['amount']),
        compensation=lambda: payment_service.refund(order['customer_id'],
order['amount'])
    )

    # Step 3: Create shipment
    saga.add_step(
        name="create_shipment",
        action=lambda: shipping_service.create_shipment(order),
        compensation=lambda: shipping_service.cancel_shipment(order['order_id'])
    )

    # Step 4: Send notification
    saga.add_step(
        name="send_notification",
        action=lambda: notification_service.send(order['customer_id'], "Order
confirmed"),
        compensation=lambda: notification_service.send(order['customer_id'],
"Order cancelled")
    )

    # Execute saga
    success = saga.execute()

    if success:
        order['status'] = 'confirmed'

```

```
else:
    order['status'] = 'failed'

return order
```

Event Sourcing Pattern

Store all changes as events:

```
from typing import List, Dict
from datetime import datetime

class EventStore:
    """
    Store for domain events (event sourcing)
    """

    def __init__(self):
        self.events = {} # In production: use database

    def append(self, aggregate_id: str, event: dict):
        """Append event to stream"""
        if aggregate_id not in self.events:
            self.events[aggregate_id] = []

        event['version'] = len(self.events[aggregate_id]) + 1
        event['timestamp'] = datetime.utcnow().isoformat()

        self.events[aggregate_id].append(event)

    def get_events(self, aggregate_id: str) -> List[dict]:
        """Get all events for aggregate"""
        return self.events.get(aggregate_id, [])

    def rebuild_aggregate(self, aggregate_id: str) -> dict:
        """Rebuild current state from events"""
        events = self.get_events(aggregate_id)
        state = {}

        for event in events:
            state = self.apply_event(state, event)

        return state

    def apply_event(self, state: dict, event: dict) -> dict:
        """Apply event to state"""
        event_type = event['event_type']

        if event_type == 'OrderCreated':
            state = {
                'order_id': event['data']['order_id'],
```

```

        'customer_id': event['data']['customer_id'],
        'amount': event['data']['amount'],
        'status': 'created',
        'items': event['data']['items']
    }

    elif event_type == 'OrderPaid':
        state['status'] = 'paid'
        state['paid_at'] = event['timestamp']

    elif event_type == 'OrderShipped':
        state['status'] = 'shipped'
        state['tracking_number'] = event['data']['tracking_number']

    elif event_type == 'OrderDelivered':
        state['status'] = 'delivered'
        state['delivered_at'] = event['timestamp']

    elif event_type == 'OrderCancelled':
        state['status'] = 'cancelled'
        state['cancellation_reason'] = event['data']['reason']

    return state

# Usage
event_store = EventStore()

# Store events
event_store.append('order-123', {
    'event_type': 'OrderCreated',
    'data': {
        'order_id': 'order-123',
        'customer_id': 'cust-456',
        'amount': 99.99,
        'items': [{'product_id': 'prod-1', 'quantity': 2}]
    }
})

event_store.append('order-123', {
    'event_type': 'OrderPaid',
    'data': {
        'payment_id': 'pay-789',
        'amount': 99.99
    }
})

event_store.append('order-123', {
    'event_type': 'OrderShipped',
    'data': {
        'tracking_number': 'TRACK-123456'
    }
})

# Rebuild current state

```

```
current_state = event_store.rebuild_aggregate('order-123')
print(current_state)
# Output: {'order_id': 'order-123', 'status': 'shipped', 'tracking_number':
'TRACK-123456', ...}

# Event log (immutable, append-only)
events = event_store.get_events('order-123')
for event in events:
    print(f"v{event['version']}: {event['event_type']} at {event['timestamp']}")
```

When to Use Event-Driven Architecture

☑ Use Event-Driven When:

- Need to decouple services
- Multiple systems need same data
- Building microservices architecture
- Need audit trail of all changes
- Want to support future use cases (new consumers)

✗ Don't Use When:

- Simple monolithic application
- Need strong consistency (use synchronous calls)
- Team lacks distributed systems expertise
- Debugging overhead not acceptable

Document continues with remaining patterns (Micro-batching, Stream Processing, Data Lake Ingestion, Real-Time Analytics, Multi-Source Aggregation, Pattern Selection Guide) in next section...

Version: 1.0

Page: 1 of 2

5. Micro-Batching Pattern {#micro-batching}

Overview

Micro-batching processes data in small batches (seconds to minutes) rather than event-by-event, providing a balance between batch efficiency and near-real-time latency.

Architecture



↓
Process batch
↓
Results

Spark Structured Streaming Implementation

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# Initialize Spark with streaming
spark = SparkSession.builder \
    .appName("MicroBatch Pipeline") \
    .config("spark.sql.streaming.schemaInference", "true") \
    .getOrCreate()

# Define schema
event_schema = StructType([
    StructField("event_id", StringType()),
    StructField("user_id", StringType()),
    StructField("event_type", StringType()),
    StructField("timestamp", TimestampType()),
    StructField("properties", MapType(StringType(), StringType()))
])

# Read stream from Kafka
events = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "kafka:9092") \
    .option("subscribe", "user-events") \
    .option("startingOffsets", "latest") \
    .load() \
    .selectExpr("CAST(value AS STRING)") \
    .select(from_json(col("value"), event_schema).alias("data")) \
    .select("data.*")

# Micro-batch aggregation (5-minute windows)
aggregated = events \
    .withWatermark("timestamp", "10 minutes") \
    .groupBy(
        window("timestamp", "5 minutes", "5 minutes"),
        "user_id",
        "event_type"
    ) \
    .agg(
        count("*").alias("event_count"),
        collect_list("event_id").alias("event_ids")
    )
```

```
# Write to data lake (micro-batches)
query = aggregated \
    .writeStream \
    .outputMode("append") \
    .format("parquet") \
    .option("path", "s3://data-lake/micro-batches/events/") \
    .option("checkpointLocation", "s3://checkpoints/events/") \
    .trigger(processingTime='5 minutes') \
    .start()

query.awaitTermination()
```

Kafka Streams Micro-Batching

```
import org.apache.kafka.streams.KafkaStreams;
import org.apache.kafka.streams.StreamsBuilder;
import org.apache.kafka.streams.kstream.*;
import java.time.Duration;

public class MicroBatchProcessor {

    public static void main(String[] args) {
        StreamsBuilder builder = new StreamsBuilder();

        // Input stream
        KStream<String, Event> events = builder.stream("user-events");

        // Micro-batch: Tumbling window of 5 minutes
        KTable<Windowed<String>, Long> counts = events
            .groupByKey()
            .windowedBy(TimeWindows.of(Duration.ofMinutes(5)))
            .count();

        // Convert to stream and write results
        counts.toStream()
            .map((windowedKey, count) -> {
                String key = windowedKey.key();
                long windowStart = windowedKey.window().start();
                long windowEnd = windowedKey.window().end();

                MicroBatchResult result = new MicroBatchResult(
                    key, count, windowStart, windowEnd
                );

                return KeyValue.pair(key, result);
            })
            .to("micro-batch-results");

        // Start Kafka Streams
        KafkaStreams streams = new KafkaStreams(builder.build(), config);
        streams.start();
    }
}
```

```
}
}
```

When to Use Micro-Batching

☑ Use Micro-Batching When:

- Need near-real-time (not sub-second)
- Want efficiency of batch processing
- Have bursty traffic patterns
- Cost optimization important

✗ Don't Use When:

- Need true real-time (<1 second)
- Data arrives at steady rate
- Simple event-by-event processing sufficient

6. Stream Processing Patterns {#stream-processing}

Pattern 6A: Windowed Aggregations

Tumbling Windows (Non-overlapping):

```
Time: 0——5——10——15——20——25
      [—W1—][—W2—][—W3—][—W4—]
```

Sliding Windows (Overlapping):

```
Time: 0——5——10——15——20——25
      [—W1—]
        [—W2—]
          [—W3—]
            [—W4—]
```

Session Windows (Activity-based):

```
Events: —●—●———●—●—●———●—
Windows:[—S1—]    [—S2—]    [S3]
        (gap timeout: 5 minutes)
```

Implementation:

```
// Flink: Tumbling Window
DataStream<Event> events = ...;

DataStream<AggregateResult> aggregated = events
    .keyBy(event -> event.userId)
    .window(TumblingEventTimeWindows.of(Time.minutes(5)))
    .aggregate(new CountAggregator());

// Flink: Sliding Window
DataStream<AggregateResult> sliding = events
    .keyBy(event -> event.userId)
    .window(SlidingEventTimeWindows.of(
        Time.minutes(10), // window size
        Time.minutes(5)   // slide interval
    ))
    .aggregate(new CountAggregator());

// Flink: Session Window
DataStream<AggregateResult> sessions = events
    .keyBy(event -> event.userId)
    .window(EventTimeSessionWindows.withGap(Time.minutes(30)))
    .aggregate(new SessionAggregator());
```

Pattern 6B: Stream Joins

Stream-Stream Join:

```
Stream A: Orders
Stream B: Payments

Join on order_id within 1-hour window
Result: Orders with payment info
```

```
// Flink stream-stream join
DataStream<Order> orders = ...;
DataStream<Payment> payments = ...;

DataStream<EnrichedOrder> enriched = orders
    .join(payments)
    .where(order -> order.orderId)
    .equalTo(payment -> payment.orderId)
    .window(TumblingEventTimeWindows.of(Time.hours(1)))
    .apply(new JoinFunction<Order, Payment, EnrichedOrder>() {
        @Override
        public EnrichedOrder join(Order order, Payment payment) {
            return new EnrichedOrder(order, payment);
        }
    });
```

Pattern 6C: Stream-Table Join

Enrich stream with reference data:

```
// Flink: Enrich events with product catalog
DataStream<ClickEvent> clicks = ...;

// Product catalog as broadcast state
MapStateDescriptor<String, Product> productStateDescriptor =
    new MapStateDescriptor<>(
        "products",
        BasicTypeInfo.STRING_TYPE_INFO,
        TypeInformation.of(Product.class)
    );

BroadcastStream<Product> productBroadcast = productStream
    .broadcast(productStateDescriptor);

DataStream<EnrichedClick> enrichedClicks = clicks
    .connect(productBroadcast)
    .process(new BroadcastProcessFunction<ClickEvent, Product, EnrichedClick>() {

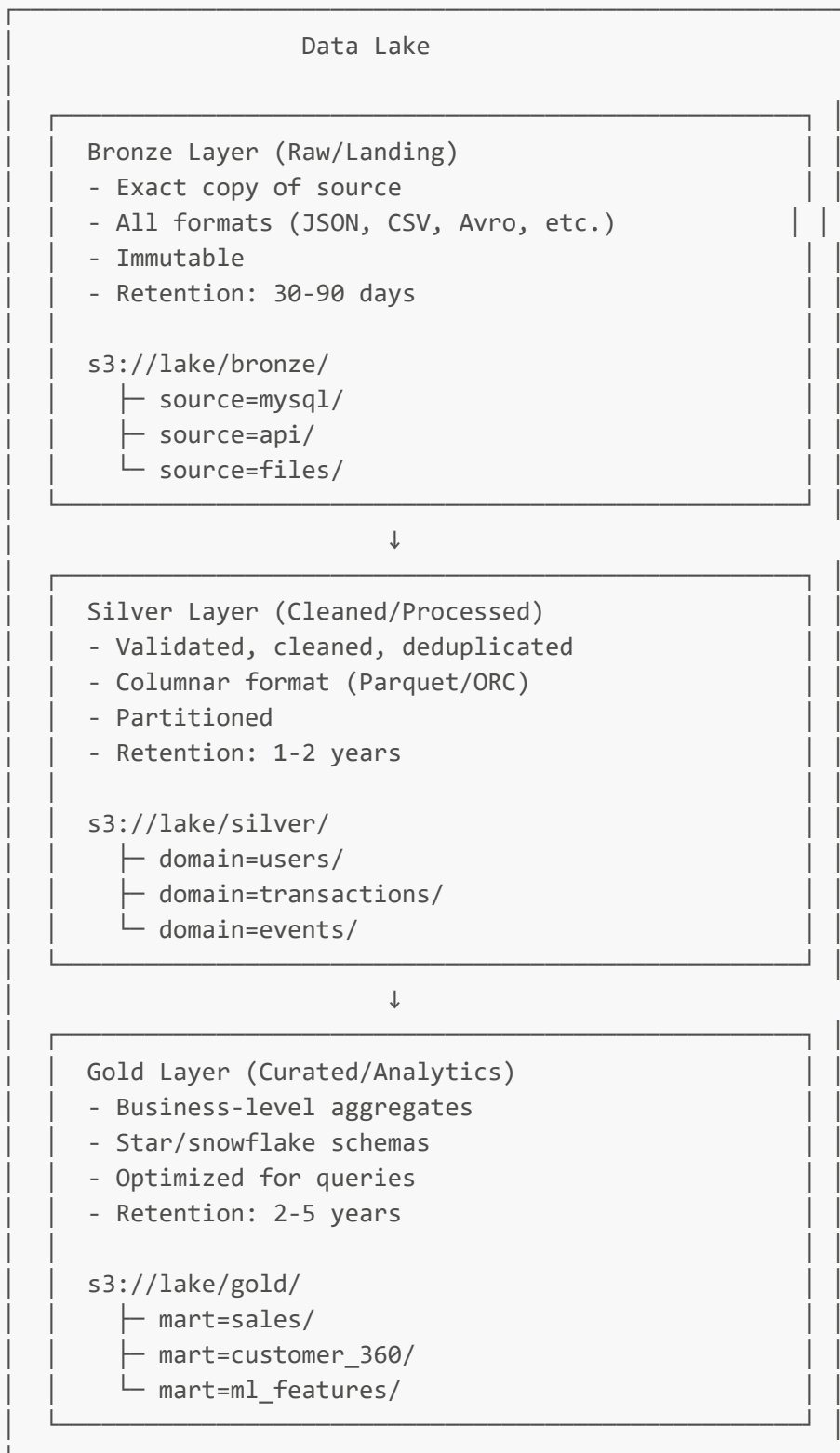
        @Override
        public void processElement(ClickEvent click,
                                   ReadOnlyContext ctx,
                                   Collector<EnrichedClick> out) {
            // Lookup product from broadcast state
            Product product = ctx.getBroadcastState(productStateDescriptor)
                .get(click.productId);

            if (product != null) {
                out.collect(new EnrichedClick(click, product));
            }
        }

        @Override
        public void processBroadcastElement(Product product,
                                             Context ctx,
                                             Collector<EnrichedClick> out) {
            // Update broadcast state when product catalog changes
            ctx.getBroadcastState(productStateDescriptor)
                .put(product.productId, product);
        }
    });
```

7. Data Lake Ingestion Patterns {#data-lake}

Pattern 7A: Landing-Processing-Curated (Medallion Architecture)



Implementation with Delta Lake:

```
from delta import *
from pyspark.sql import SparkSession

# Initialize Spark with Delta
spark = SparkSession.builder \
    .appName("Data Lake Pipeline") \
```

```

        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension") \
        .config("spark.sql.catalog.spark_catalog",
"org.apache.spark.sql.delta.catalog.DeltaCatalog") \
        .getOrCreate()

# Bronze Layer: Raw ingestion
def ingest_to_bronze(source_data, source_name):
    """Ingest raw data to bronze layer"""
    bronze_path = f"s3://lake/bronze/source={source_name}/"

    source_data.write \
        .format("delta") \
        .mode("append") \
        .partitionBy("ingestion_date") \
        .save(bronze_path)

# Silver Layer: Clean and validate
def process_to_silver(bronze_path, silver_path):
    """Clean and validate data"""
    # Read from bronze
    bronze_df = spark.read.format("delta").load(bronze_path)

    # Data quality checks and cleaning
    silver_df = bronze_df \
        .filter(col("id").isNotNull()) \
        .filter(col("amount") > 0) \
        .dropDuplicates(["id"]) \
        .withColumn("processed_at", current_timestamp())

    # Write to silver
    silver_df.write \
        .format("delta") \
        .mode("overwrite") \
        .partitionBy("date") \
        .option("mergeSchema", "true") \
        .save(silver_path)

# Gold Layer: Business aggregates
def create_gold_mart(silver_path, gold_path):
    """Create business-level aggregates"""
    silver_df = spark.read.format("delta").load(silver_path)

    # Business logic
    gold_df = silver_df \
        .groupBy("customer_id", "date") \
        .agg(
            sum("amount").alias("total_spend"),
            count("*").alias("transaction_count"),
            avg("amount").alias("avg_transaction")
        )

    # Write to gold
    gold_df.write \
        .format("delta") \

```

```
.mode("overwrite") \
.partitionBy("date") \
.save(gold_path)
```

Pattern 7B: Time-Travel & Versioning

```
# Delta Lake time travel
# Read historical version
df_yesterday = spark.read \
    .format("delta") \
    .option("versionAsOf", 42) \
    .load("s3://lake/silver/transactions/")

# Read at specific timestamp
df_last_week = spark.read \
    .format("delta") \
    .option("timestampAsOf", "2026-08-08") \
    .load("s3://lake/silver/transactions/")

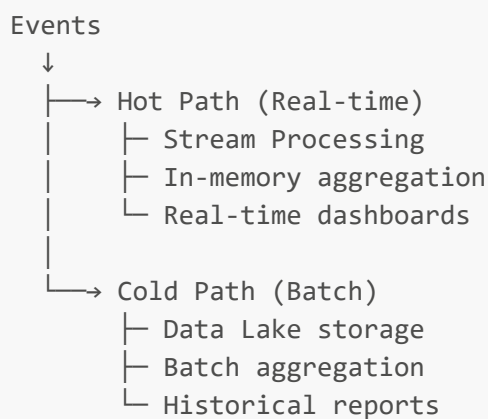
# View history
delta_table = DeltaTable.forPath(spark, "s3://lake/silver/transactions/")
history = delta_table.history()
history.show()

# Restore to previous version
delta_table.restoreToVersion(40)

# Vacuum old files (remove old versions)
delta_table.vacuum(retentionHours=168) # 7 days
```

8. Real-Time Analytics Pipelines {#realtime-analytics}

Pattern 8A: Hot Path / Cold Path



Implementation:

```

# Hot path: Real-time metrics
def hot_path_pipeline():
    """Process for real-time dashboards"""
    events = spark \
        .readStream \
        .format("kafka") \
        .load()

    # Real-time aggregation (last 5 minutes)
    real_time_metrics = events \
        .withWatermark("timestamp", "1 minute") \
        .groupBy(
            window("timestamp", "5 minutes"),
            "metric_name"
        ) \
        .agg(
            avg("value").alias("avg_value"),
            max("value").alias("max_value"),
            min("value").alias("min_value")
        )

    # Write to Redis for real-time dashboard
    real_time_metrics \
        .writeStream \
        .foreachBatch(lambda df, epoch: write_to_redis(df)) \
        .start()

# Cold path: Historical storage
def cold_path_pipeline():
    """Store for historical analysis"""
    events = spark \
        .readStream \
        .format("kafka") \
        .load()

    # Write to data lake (Parquet)
    events \
        .writeStream \
        .format("parquet") \
        .option("path", "s3://lake/raw/events/") \
        .option("checkpointLocation", "s3://checkpoints/events/") \
        .partitionBy("date") \
        .trigger(processingTime="10 minutes") \
        .start()

```

Pattern 8B: Real-Time Feature Store

```

from feast import FeatureStore, Entity, FeatureView, Field
from feast.types import Float32, Int64, String
from datetime import timedelta

```

```

# Define entity
user = Entity(
    name="user",
    join_keys=["user_id"],
    description="User entity"
)

# Define feature view (real-time features)
user_realtime_features = FeatureView(
    name="user_realtime_features",
    entities=[user],
    ttl=timedelta(hours=1),
    schema=[
        Field(name="last_purchase_amount", dtype=Float32),
        Field(name="purchases_last_hour", dtype=Int64),
        Field(name="current_session_duration", dtype=Int64)
    ],
    online=True,
    source=PushSource(
        name="user_realtime_push_source",
        batch_source=None
    )
)

# Flink job to update feature store in real-time
class FeatureStoreUpdater(ProcessFunction):

    def processElement(self, event, ctx, out):
        """Update feature store with real-time features"""
        user_id = event.user_id

        # Calculate features
        features = {
            "user_id": user_id,
            "last_purchase_amount": event.amount,
            "purchases_last_hour": self.count_purchases_last_hour(user_id),
            "current_session_duration": self.get_session_duration(user_id)
        }

        # Push to feature store
        feature_store.push(
            "user_realtime_features",
            pd.DataFrame([features])
        )

        out.collect(features)

# ML model serving using feature store
def predict(user_id):
    """Get prediction using real-time features"""
    # Fetch features from online store
    features = feature_store.get_online_features(
        features=[

```

```
        "user_realtime_features:last_purchase_amount",
        "user_realtime_features:purchases_last_hour",
        "user_batch_features:total_lifetime_value"
    ],
    entity_rows=[{"user_id": user_id}]
).to_dict()

# Make prediction
prediction = model.predict([features])
return prediction
```

9. Multi-Source Aggregation {#multi-source}

Pattern 9A: Unified Data Integration

Scenario: Aggregate data from 10+ sources into single view



(Data Lake + Warehouse)

Implementation:

```
class MultiSourceAggregator:
    """
    Aggregate data from multiple sources
    """

    def __init__(self, spark: SparkSession):
        self.spark = spark

    def read_mysql_orders(self):
        """Read from MySQL"""
        return self.spark.read \
            .format("jdbc") \
            .option("url", "jdbc:mysql://host:3306/db") \
            .option("dbtable", "orders") \
            .load()

    def read_postgres_users(self):
        """Read from PostgreSQL"""
        return self.spark.read \
            .format("jdbc") \
            .option("url", "jdbc:postgresql://host:5432/db") \
            .option("dbtable", "users") \
            .load()

    def read_mongodb_products(self):
        """Read from MongoDB"""
        return self.spark.read \
            .format("mongo") \
            .option("uri", "mongodb://host:27017/db.products") \
            .load()

    def read_s3_logs(self):
        """Read from S3"""
        return self.spark.read \
            .json("s3://logs/app-logs/*/")

    def read_api_data(self, api_name):
        """Read from API (via Kafka topic)"""
        return self.spark.read \
            .format("kafka") \
            .option("kafka.bootstrap.servers", "kafka:9092") \
            .option("subscribe", f"api.{api_name}") \
            .load()

    def aggregate_customer_360(self):
        """Create unified customer view"""
        # Read from all sources
```

```

orders = self.read_mysql_orders()
users = self.read_postgres_users()
products = self.read_mongodb_products()
analytics = self.read_api_data("google_analytics")
crm = self.read_api_data("salesforce")
support = self.read_api_data("zendesk")

# Join everything
customer_360 = users \
    .join(orders, "user_id", "left") \
    .join(products, "product_id", "left") \
    .join(analytics, "user_id", "left") \
    .join(crm, ["email"], "left") \
    .join(support, ["email"], "left")

# Aggregate
unified = customer_360.groupBy("user_id").agg(
    first("user_name").alias("name"),
    first("email").alias("email"),
    sum("order_amount").alias("total_spend"),
    count("order_id").alias("total_orders"),
    max("order_date").alias("last_purchase_date"),
    first("crm_account_id").alias("crm_id"),
    count("support_ticket_id").alias("support_tickets"),
    collect_list("product_category").alias("purchased_categories")
)

return unified

```

Pattern 9B: Schema Evolution Across Sources

```

from pyspark.sql.types import *

class SchemaEvolutionHandler:
    """
    Handle schema changes across multiple sources
    """

    def __init__(self, schema_registry_url: str):
        self.schema_registry = SchemaRegistry(schema_registry_url)

    def read_with_schema_evolution(self, source: str, table: str):
        """Read with automatic schema evolution"""
        # Get latest schema from registry
        latest_schema = self.schema_registry.get_latest_schema(f"{source}.
{table}")

        # Read data
        df = self.spark.read \
            .format("avro") \
            .option("avroSchema", latest_schema) \

```

```
        .load(f"s3://lake/raw/{source}/{table}/")

    # Handle missing columns (backward compatibility)
    for field in latest_schema.fields:
        if field.name not in df.columns:
            df = df.withColumn(field.name, lit(None).cast(field.dataType))

    # Handle removed columns (forward compatibility)
    for col in df.columns:
        if col not in [f.name for f in latest_schema.fields]:
            df = df.drop(col)

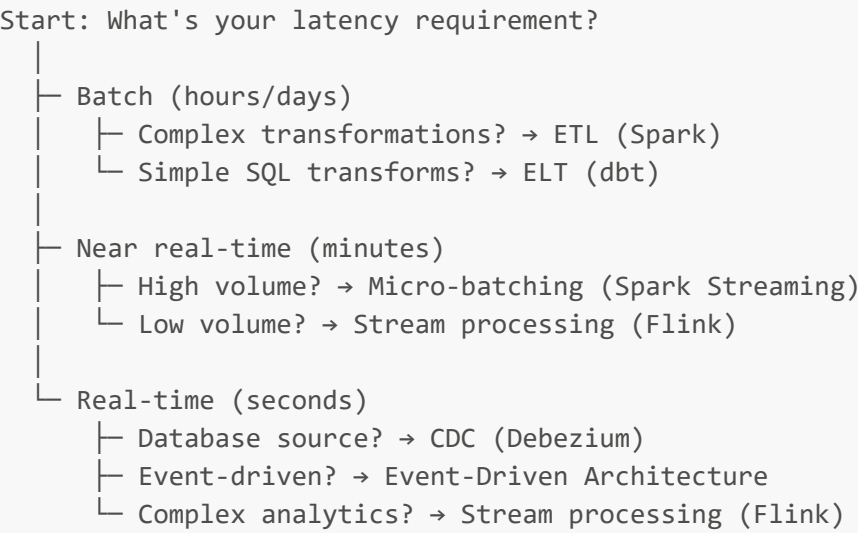
    return df
```

10. Pattern Selection Guide {#pattern-selection}

Decision Matrix

Requirement	Recommended Pattern	Alternative
Real-time DB replication	CDC (Debezium)	API polling (if CDC not available)
Batch analytics	ETL with Spark	ELT with dbt
Event-driven microservices	Event-Driven + Kafka	API calls (synchronous)
Near real-time (minutes)	Micro-batching	Stream processing
Sub-second latency	Stream processing (Flink)	Custom in-memory
Data lake ingestion	Medallion (Bronze/Silver/Gold)	Single-layer lake
Real-time dashboards	Hot/Cold path	Stream processing only
Multi-source integration	Unified aggregation	Individual pipelines

Selection Flowchart



Pattern Combinations

Common Combinations:

1. CDC + Stream Processing + Data Lake

- Real-time database replication
- Stream transformations
- Historical storage

2. Event-Driven + Micro-batching + Warehouse

- Microservices emit events
- Batch processing for analytics
- Load to warehouse

3. Multi-source + ETL + ELT

- Integrate multiple sources
- Pre-process in ETL
- Final transforms in warehouse

Conclusion

Data ingestion patterns provide proven solutions to common challenges. Key takeaways:

1. **Choose Based on Requirements:** No one-size-fits-all solution
2. **Start Simple:** Begin with batch, evolve to streaming as needed
3. **Combine Patterns:** Most production systems use multiple patterns
4. **Monitor & Optimize:** Continuously measure and improve
5. **Plan for Evolution:** Schema changes, scaling, new sources

Next Steps

- **Technology Stack:** [Document 03](#) - Compare tools for each pattern
- **Implementation Guide:** [Document 04](#) - Step-by-step implementations
- **Case Studies:** [Document 07](#) - Real-world examples

Document Complete Version: 1.0 **Total Pages:** 2 of 2